Introduction to Artificial Intelligence

Lecture 1 · What is AI? Problem solving as search

Search problems · blind search

How you will be assessed

No final exam

Your grade comes from the work you do during the course.

Team assignments

Three programming assignments, done in teams, spread over the semester.

Defense

Each team presents and defends its work in person.

You learn by building the algorithms, and you show what you learned by explaining them, and making adaptations/modifications, in person during the defense.

Three team assignments

Assignment	Covers	Released	Due
1 · Search and logic programming	weeks 1–6	week 3	week 5
2 · Games, MDPs and reinforcement learning	weeks 7–10	week 8	week 10
3 · Machine learning	weeks 11–13	week 11	week 13

- Teams work on the assignments, **implement** algorithms, test and analyze them.

Dates are tentative; they will be confirmed on moodle.

The defense

Each team presents and defends its work in person. Every member should be able to

- **explain** any part of the code, and the design decisions behind it;
- **adapt** the code to a small new request made during the defense, or explain how they would add that behavior to their implementation;
- **trace** an algorithm by hand on a small example;
- **sketch** one of the proofs seen in class.

> Every member must understand **all** of the work, not just the part they wrote: the grade depends on the work and on how well each member explains it.

Today

01	**What is AI?**	Definitions, a short history, the main areas
02	**Agents**	Environments and problem-solving agents
03	**Search problems**	Formulation, examples, and the mathematics behind them
04	**Trees, graphs and frontiers**	How a search explores the state space
05	**Blind search**	Breadth-first, uniform-cost, depth-first, depth-limited, iterative deepening, bidirectional
06	**Search problems in Python**	An object-oriented implementation of all of the above

01

What is AI?

AI is more than machine learning

- Today "AI" usually means machine learning, and often just large language models.
- This course takes the broader view: **AI studies agents that act well in their environment.**

Symbolic AI

Search, logic, planning, constraints.
Knowledge is explicit.

Statistical AI

Probability, learning from data.
Knowledge is learned.

The most impressive systems combine both. **AlphaGo** = tree search + neural networks + reinforcement learning.

Four ways to define AI

	Humanly	Rationally
Thinking	Cognitive modeling: how do people think?	Laws of thought: logic, correct inference
Acting	The Turing test: behave like a person	Rational agents: do the right thing

- *Humanly* measures a system against people; *rationally*, against an ideal performance measure.
- Most of this course: **acting rationally**. It is general (logic is one tool among several), and it can be analyzed mathematically.

Acting humanly: the Turing test (1950)

Turing's *Computing Machinery and Intelligence* replaced "Can machines think?" by an operational question: **can a machine pass for a human in a written conversation?**

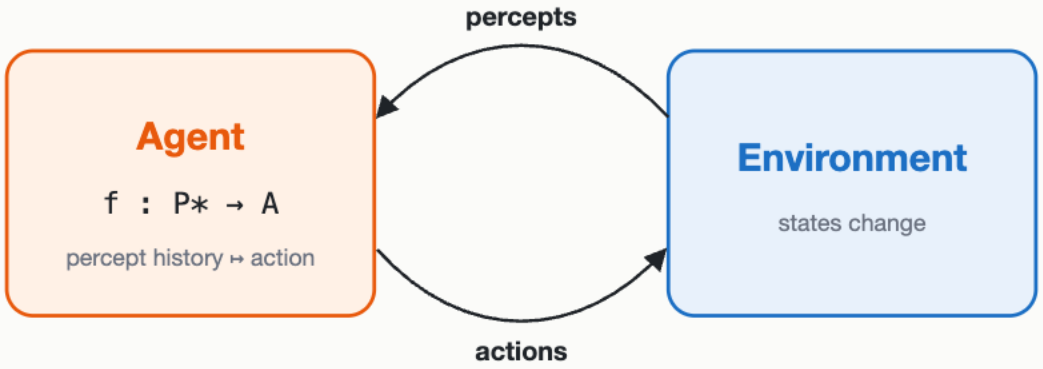
Passing requires

- natural language processing, knowledge representation, automated reasoning, machine learning;
- and, for the *total* Turing test, computer vision and robotics.

The test remains an important reference point. Most AI research, though, studies the general principles of intelligent behavior rather than imitating people, much as aeronautics studied the principles of flight rather than imitating birds.

Rational agents

- An **agent** perceives its environment and acts on it.
- Abstractly, an agent is a function $f : P* \to A$ from percept histories to actions.
- A **performance measure** scores the resulting sequence of environment states.
- A **rational** agent chooses the action that maximizes the expected performance, given its percepts and knowledge.
- Perfect rationality is usually too expensive to compute: we aim for **bounded rationality**.

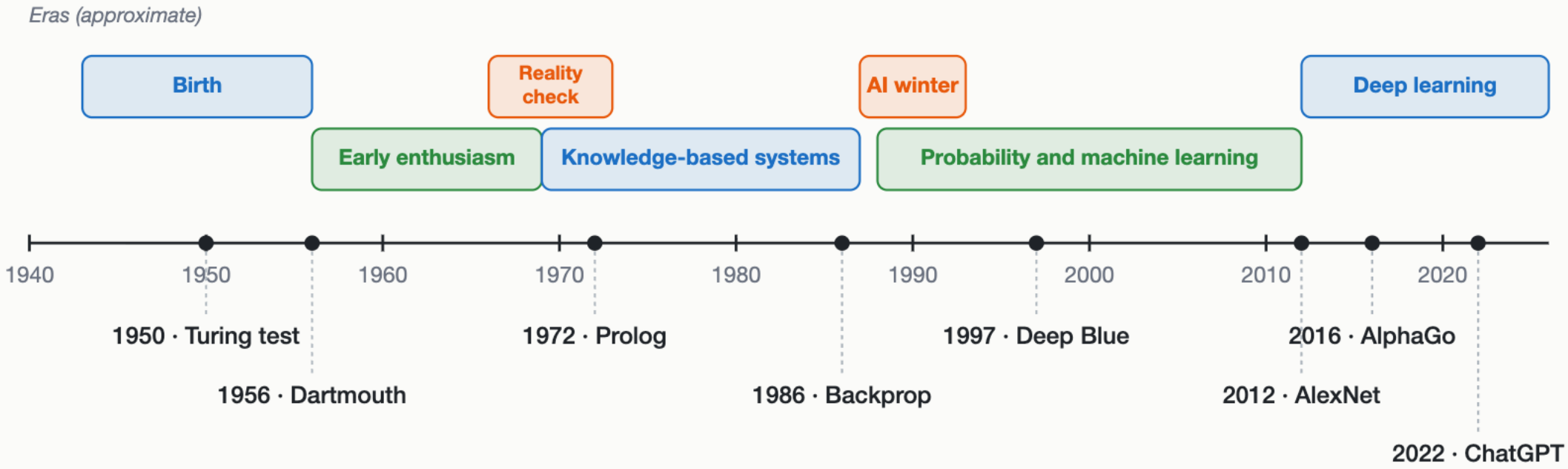


Where AI comes from

Field	Contributions
Mathematics	logic (Boole, Frege, Gödel), computability (Turing), complexity and NP-completeness (Cook, Karp), probability (Bayes), dynamic programming (Bellman)
Philosophy	reasoning, the mind as a physical system, the foundations of learning
Economics	utility, decision theory, game theory
Neuroscience, psychology	how brains and people process information
Computer engineering	the machines themselves
Control theory	feedback, optimal control
Linguistics	grammar, meaning, knowledge representation

Much of AI is applied mathematics: **logic, combinatorics, probability and optimization**.

A short history



A recurring pattern: optimism, then a wall (**combinatorial explosion**, lack of data or computing power), then new techniques.

Milestones, and where they meet this course

Year	System	Main techniques	In this course
1997	Deep Blue defeats Kasparov at chess	adversarial search, evaluation functions	week 7
2012	AlexNet wins ImageNet	deep neural networks	week 13
2016	AlphaGo defeats Lee Sedol at Go	tree search + neural networks + reinforcement learning	weeks 8, 10, 13
2020	AlphaFold 2 predicts protein structures	deep learning (Nobel Prize in Chemistry, 2024)	—
2022	ChatGPT	transformers + reinforcement learning from human feedback	weeks 10, 13
2024	AlphaProof: silver-medal level at the International Mathematical Olympiad	search for formal proofs in Lean, guided by learning	weeks 1–8, 10
2025	Gold-medal level at the International Mathematical Olympiad	reasoning language models	—

Search did not disappear: it is still at the core of game playing, planning and theorem proving.

The main areas of AI

- **Search and planning**: finding sequences of actions ★
- **Constraint satisfaction**: scheduling, puzzles, configuration ★
- **Games and multi-agent systems** ★
- **Knowledge representation and reasoning**: logic, ontologies (★ via Prolog)
- **Reasoning under uncertainty**: probability, sequential decisions ★
- **Machine learning**: learning from data ★
- **Natural language processing**
- **Computer vision**
- **Robotics**

★ *covered in this course*

This course

Part	Weeks	Topics
I · Search	1–8	blind search, logic programming, heuristic search, local search and genetic algorithms, constraints, games
II · Uncertainty	9–10	Markov decision processes, reinforcement learning
III · Learning	11–13	fundamentals, k-nearest neighbors, decision trees, linear models, neural networks

Running threads:

- we **implement** the algorithms ourselves, and use libraries only for larger problems;
- we **prove** what the algorithms guarantee;
- we practice **programming paradigms**: object-oriented, logic and functional programming.

02

Agents and problem solving

Properties of task environments

Property	8-puzzle	Chess with a clock	Taxi driving
Fully observable?	yes	yes	no
Deterministic?	yes	yes	no
Static?	yes	semi	no
Discrete?	yes	yes	no
Single agent?	yes	no	no
Rules known?	yes	yes	mostly

We start with the easiest column: fully observable, deterministic, static, discrete, single-agent, known. In that setting, problem solving is **search**.

Problem-solving agents

If the environment is fully observable, deterministic, discrete and known, a solution is a **fixed sequence of actions**, computed before acting.

1. **Goal formulation**: what do we want to achieve?
2. **Problem formulation**: an **abstraction**. Which states and actions matter, and which details can we ignore?
3. **Search**: simulate sequences of actions until one reaches the goal.
4. **Execution**: carry out the actions, without needing to look (*open loop*).

An abstraction is useful if every abstract solution can be carried out in the real world, and it is much simpler than the world.

03

Search problems

The water jug problem

> We have two jugs, holding **5 liters** and **3 liters**, and an unlimited supply of water. The jugs have no markings. How can we measure **exactly 4 liters**?

Allowed moves:

- **fill** a jug from the tap;
- **empty** a jug;
- **pour** one jug into the other, until the first is empty or the second is full.

Is there a solution? What is the shortest one? How would a program find it, and how could we be sure it is the shortest?

A solution

Step	Action	jug 1	jug 2
0	start	0	0
1	fill jug 1	5	0
2	pour jug 1 into jug 2	2	3
3	empty jug 2	2	0
4	pour jug 1 into jug 2	0	2
5	fill jug 1	5	2
6	pour jug 1 into jug 2	4	3

Six moves. Is there a shorter solution?

Search problems

Definition. A *search problem* consists of

- a set of **states** S, and an **initial state** $s_0 \in S$;
- for each state s, a set of **applicable actions** $A(s)$;
- a **transition model**: $T(s, a) \in S$ is the state reached by doing a in s;
- a **goal test**: a set of goal states $G \subseteq S$;
- an **action cost** $c(s, a, s') > 0$, often 1.

A **solution** is a sequence of actions $a_1, \ldots, a_k$ leading from s_0 to a state in G. It is **optimal** if its total cost is minimal among all solutions.

The state space is a graph

A search problem defines a directed graph on S, with edges labeled by actions and weighted by costs:

$$E = \{ (s,\ T(s,a)) \ :\ s \in S,\ a \in A(s) \}$$

- Solutions are paths from s_0 to G; optimal solutions are shortest (cheapest) paths.
- So far this is graph theory. The difference: the graph is **implicit**. We only have s_0 and a successor function, and the graph is usually too large to build, or **infinite**.
- **Search** will explore this graph lazily, generating states only when needed.

The water jug problem, formally

- **States**: pairs (x, y) with $0 \le x \le 5$ and $0 \le y \le 3$, so 24 pairs.
- **Initial state** (0, 0). **Goal**: x = 4 or y = 4.
- **Transitions**, with capacities a = 5 and b = 3:

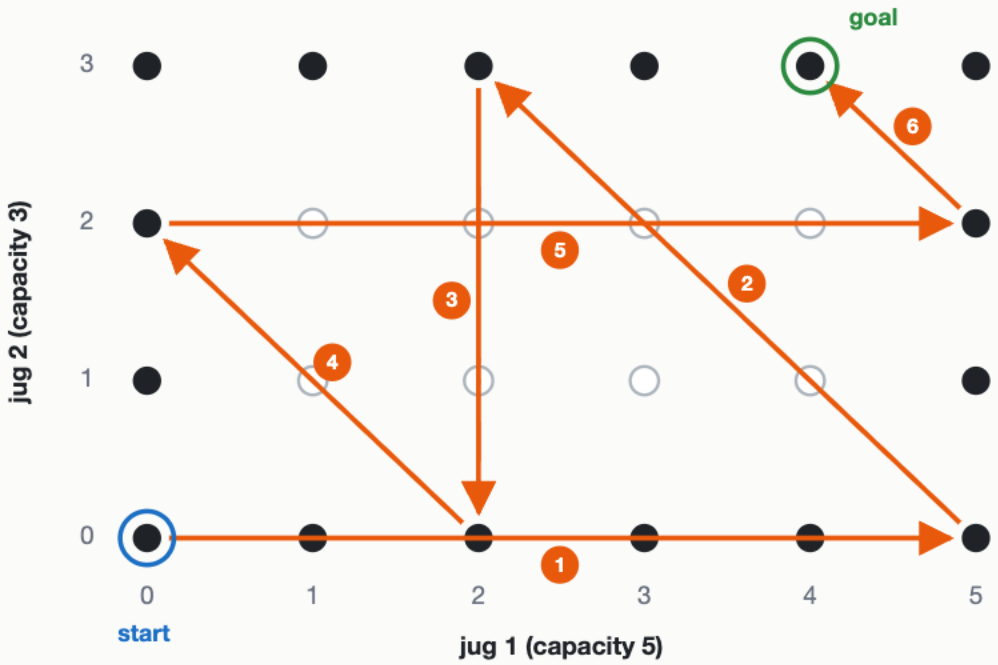
Action	Resulting state
fill jug 1, fill jug 2	(a, y), (x, b)
empty jug 1, empty jug 2	(0, y), (x, 0)
pour jug 1 into jug 2	(x − t, y + t), where t = min(x, b − y)
pour jug 2 into jug 1	(x + t′, y − t′), where t′ = min(y, a − x)

- **Cost**: 1 per action.

The state space, drawn

- Each point is a state (x, y).
- Filling and emptying move horizontally or vertically, to the border.
- Pouring moves along a **diagonal** x + y = const.
- Only the **16 border points** are reachable from (0, 0); the 8 interior points never are.

Why?



A mathematical aside: invariants

Claim 1. In every reachable state, at least one jug is empty or full.

Proof. True for (0, 0). Filling makes a jug full, emptying makes it empty, and pouring stops exactly when the source is empty or the target is full. ∎

Claim 2. Let $d = \gcd(a, b)$. In every reachable state, $d \mid x$ and $d \mid y$.

Proof. Induction on the length of the path. Filling and emptying produce a, b or 0. Pouring moves $t = \min(x, b - y)$ liters, a multiple of d. ∎

So with jugs of 6 and 4 liters you can **never** measure 3 liters: every search will fail.

What search can and cannot tell us

- In a **finite** state space, a **systematic search** either finds a solution or visits every reachable state. A search that fails is a **proof of impossibility** for that instance.
- Mathematics explains the whole family at once. With capacities a and b, you can measure exactly the multiples of gcd(a, b) up to max(a, b).
- The theorem says a solution **exists**; **search finds sequences of actions leading to solutions**.

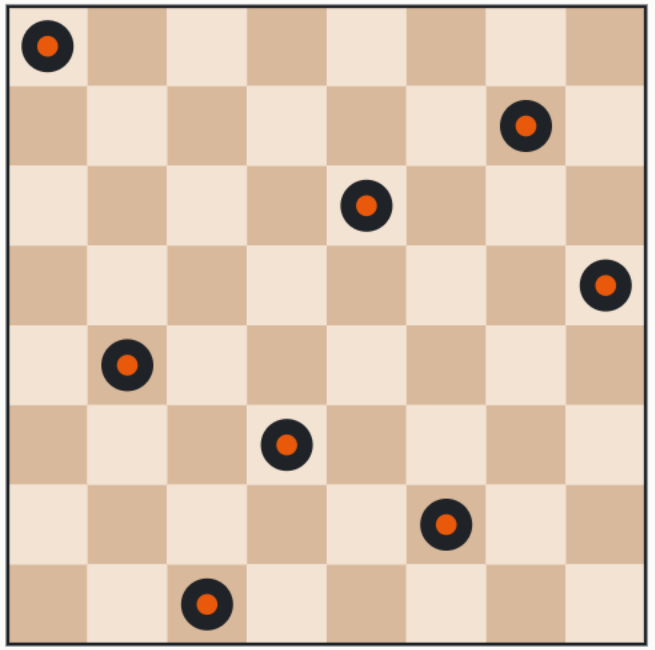
Search and proof complement each other.

Choosing the formulation: 8 queens

Place 8 queens on a chessboard so that no two attack each other.

- **Formulation A**: add a queen to any empty square, one at a time. $64 \cdot 63 \cdots 57 \approx 1.8 \times 10^{14}$ sequences to consider.
- **Formulation B**: one queen per column, in any of the 8 rows of its column. $8^8 \approx 1.7 \times 10^7$ configurations.

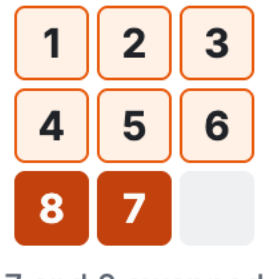
Same problem, seven orders of magnitude apart. Choosing states and actions is a crucial modeling decision.



The 8-puzzle

Eight numbered tiles and a blank square on a 3 × 3 board.





- **States**: the arrangements of the tiles and the blank, 9! = 362,880 of them.
- **Actions**: move the blank up, down, left or right, by sliding a tile into it.
- **Goal**: tiles 1 to 8 in order, with the blank in the bottom-right corner.
- Not every arrangement is solvable: on the right, the goal with 7 and 8 swapped.

Search problems beyond puzzles

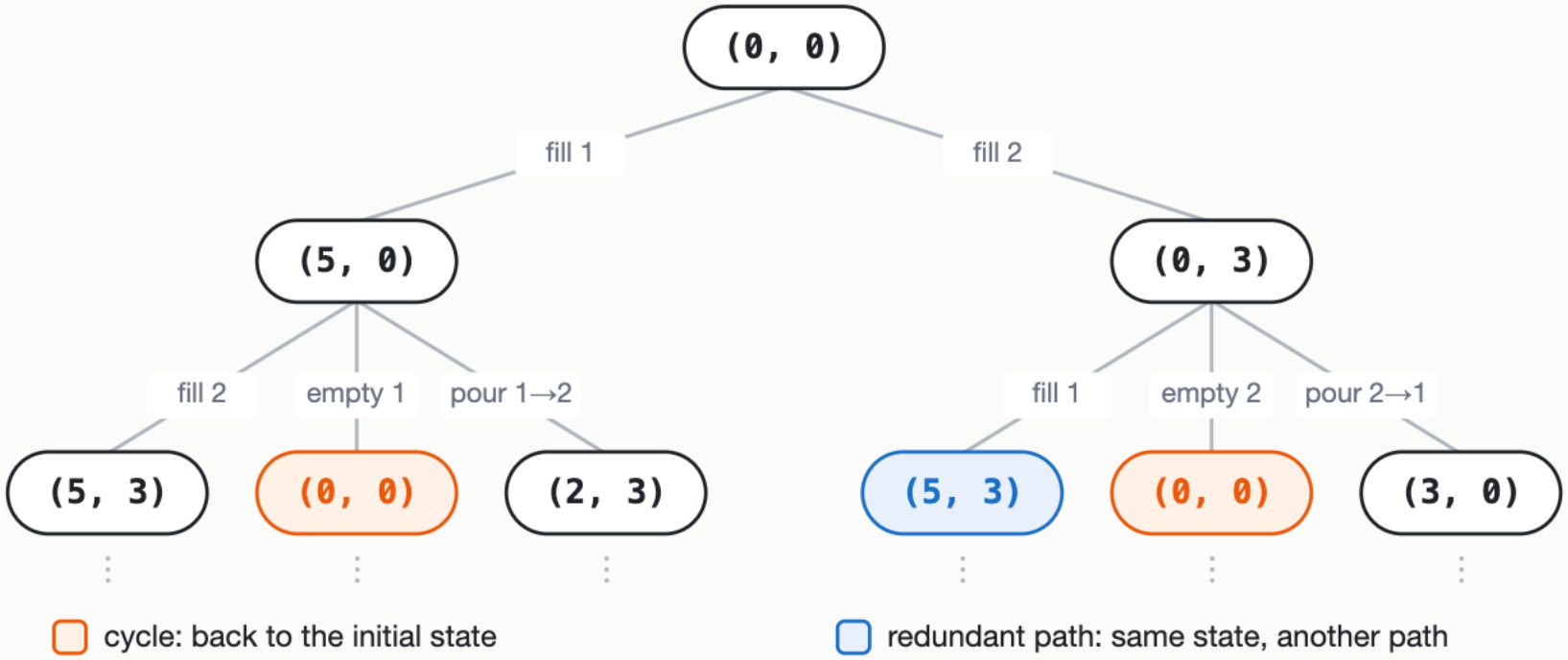
Problem	States	Actions	Goal
Route finding (GPS)	intersections of a road map	drive along a road segment	reach the destination in the least time
Trip planning	airport, date and time	take a flight that departs later than current time/date	arrive by a deadline, at the lowest price
Robot motion	position and joint angles of the robot	small movements of its joints	reach a target pose without collisions
Logistics	where each truck and package is	load, drive or unload	every package delivered
Software verification	All possible assignments to program variables	execute current statement (& change state accordingly)	a state that breaks an intended property: a bug
Social networks	people	follow a friendship link	reach a given person through the fewest links

Whenever a solution is a **sequence of decisions**, search can find it.

04

Search trees, graphs and frontiers

The search tree



The root holds the initial state; the children of a node hold its successors. A **finite** state space can produce an **infinite** search tree: (0, 0) → (5, 0) → (0, 0) → ⋯

States and nodes

State

A configuration of the problem, for example the jug contents (2, 3).

The states and actions form the state space: a graph.

Node

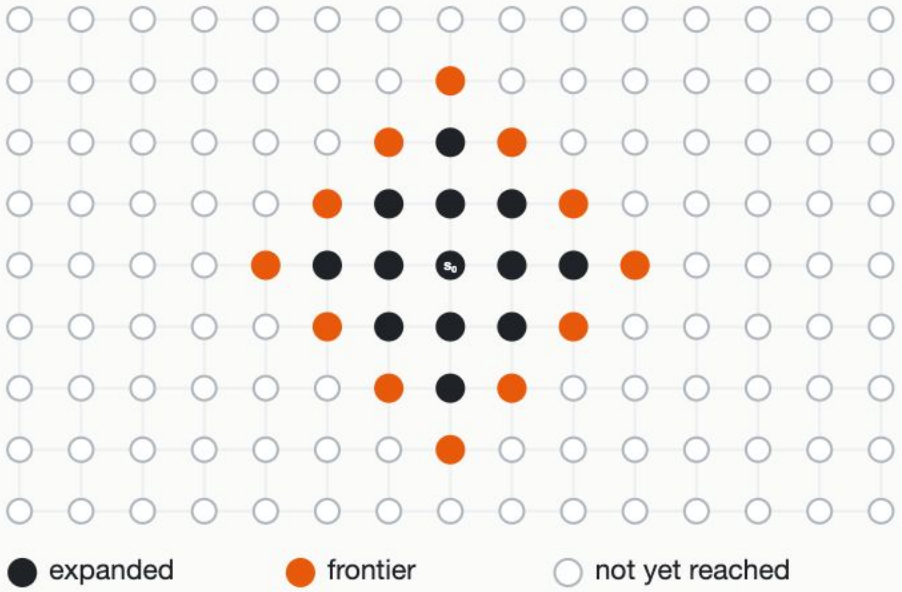
A data structure of the search, which records

- the **state** it holds;
- its **parent** node;
- the **action** that produced it;
- the **path cost** g from the root;
- its **depth**.

Two nodes may hold the same state, reached by different paths. Following the parent links back from a goal node gives the solution.

The frontier

- **Frontier**: nodes generated but not yet expanded.
- **Reached**: states already seen.
- Every path from s_0 to an unreached state passes through the frontier: the frontier **separates** the explored part of the graph from the rest.
- A search **strategy** is a rule for choosing which frontier node to expand next.



One generic algorithm

```
function GRAPH-SEARCH(problem, frontier):
    node ← NODE(problem.initial-state)
    frontier.add(node)
    reached ← {node.state: node}
    while frontier is not empty:
        node ← frontier.remove()
        if problem.is-goal(node.state): return node
        for each child in EXPAND(problem, node):
            s ← child.state
            if s not in reached or child.path-cost < reached[s].path-cost:
                reached[s] ← child
                frontier.add(child)
    return failure
```

- With **reached**, this is **graph search**; without it, **tree-like search**.
- The cost test matters only when actions have different costs (uniform-cost search).

The frontier is the strategy

The same algorithm with a different frontier is a different search strategy:

Frontier	Removed first	Strategy
FIFO queue	the oldest node	breadth-first search
LIFO stack	the newest node	depth-first search
priority queue ordered by g	the cheapest node	uniform-cost search

How to compare strategies

- **Completeness**: if a solution exists, is one always found? If none exists, is failure reported?
- **Cost optimality**: does it find a solution of minimal cost?
- **Time complexity**: how many nodes are generated in the worst case?
- **Space complexity**: how many nodes need to be simultaneously stored in the worst case?

Symbol	Meaning
b	branching factor: maximum number of successors of a state
d	depth of the shallowest solution
m	maximum depth of the state space (possibly ∞)
C∗, ε	cost of an optimal solution; minimum action cost

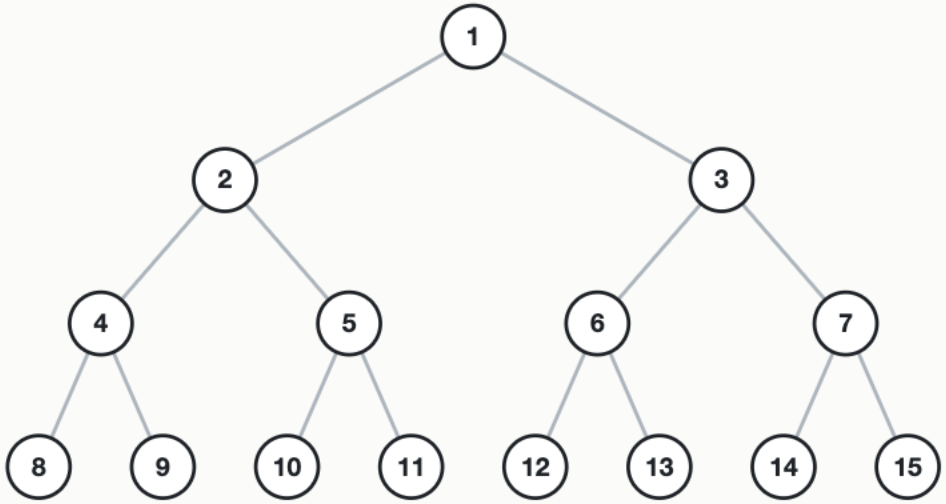
05

Blind search

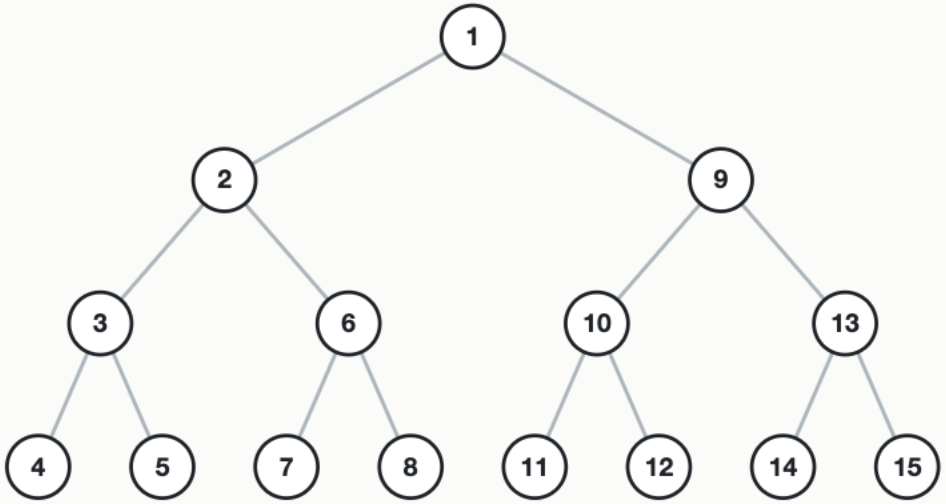
Uninformed (blind) search

- Uses **only the problem definition**: no estimate of how close a state is to a goal.
- Strategies differ only in the **order** in which they expand nodes. The numbers show that order:

Breadth-first: level by level (FIFO queue)



Depth-first: deepest first (LIFO stack)



Breadth-first search

Expand the **shallowest** unexpanded node: the frontier is a FIFO queue.

On the water jug problem, breadth-first search returns

`(0, 0)` → `(5, 0)` → `(2, 3)` → `(2, 0)` → `(0, 2)` → `(5, 2)` → **`(4, 3)`**

- **6 moves**: the minimum, as the theorem on the next slide guarantees.
- Depth-first search returns a solution with 8 moves.
- With jugs of 6 and 4 liters and target 3, both explore every reachable state and report **failure**.

Breadth-first search: guarantees

> **Theorem.** If b is finite and a solution exists, BFS finds one. If all actions have the same cost, the solution it finds is optimal.

Proof sketch.

1. By induction on k: every node of depth k leaves the queue before any node of depth k + 1, since the queue holds the nodes of depth k followed by their children.
2. Each state is first reached through a path of minimal length, so **reached** never discards a shortest path.
3. With finite b there are finitely many nodes of depth ≤ d. The shallowest goal, of depth d, is removed from the frontier after finitely many steps, before any deeper node. Minimal depth means minimal cost when all costs are equal. ∎

Breadth-first search: the price

Nodes generated up to depth d:

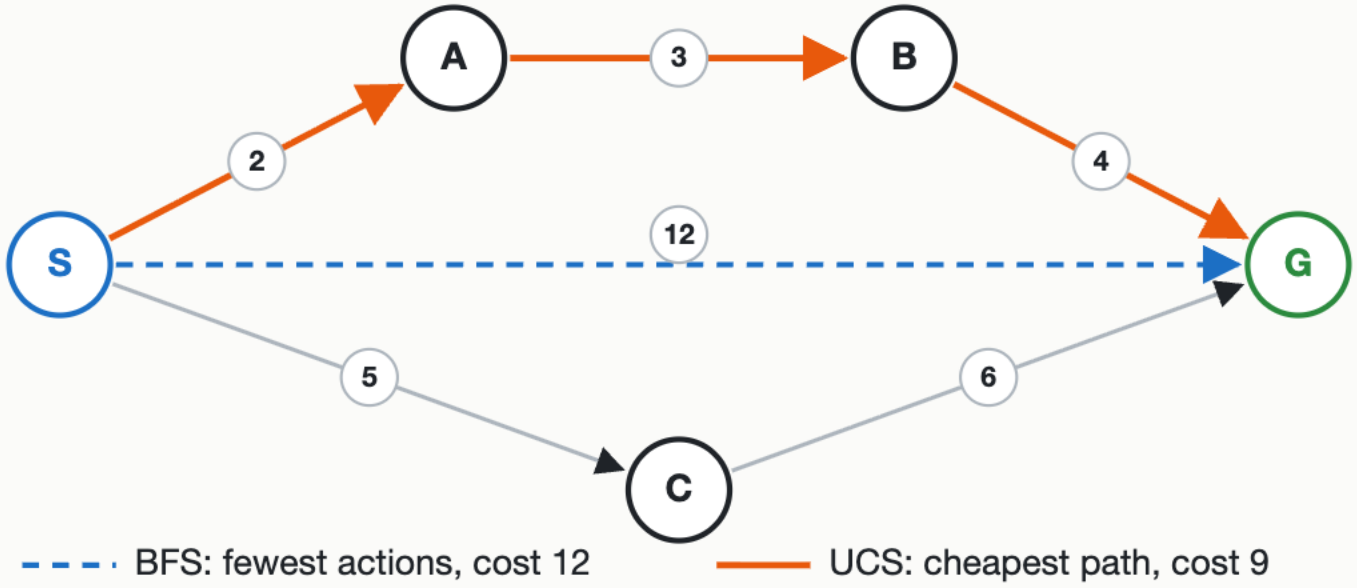
$$1 + b + b^2 + \cdots + b^d = \frac{b^{d+1} - 1}{b - 1} = O(b^d)$$

Every generated node stays in memory, so time **and space** are $O(b^d)$. With b = 10, a million nodes per second and 1 kB per node:

Depth	Nodes	Time	Memory
6	1.1×10^6	1.1 s	1.1 GB
8	1.1×10^8	2 min	111 GB
10	1.1×10^{10}	3 h	11 TB
12	1.1×10^{12}	13 days	1.1 PB
14	1.1×10^{14}	3.5 years	111 PB

Memory runs out long before time does.

Different costs: uniform-cost search



- BFS minimizes the **number** of actions, not their **cost**.
- **Uniform-cost search** expands the node with the lowest path cost g(n): the frontier is a priority queue. It is similar to **Dijkstra's algorithm**, but run on an implicit graph.

Uniform-cost search: guarantees

Test for the goal when a node is **removed** from the frontier, not when it is generated: G is first generated with g = 12, and only later with g = 9.

> **Theorem.** If every action costs at least ε > 0 and b is finite, UCS is complete and cost-optimal.

Proof sketch. Costs are positive, so nodes leave the queue in nondecreasing order of g. When a goal node n is removed, every node in the frontier has g ≥ g(n). Any other path to a goal passes through the frontier, so it costs at least g(n). For completeness, only finitely many paths cost less than C* + ε, because each action adds at least ε. ∎

Time and space: $O\left(b^{1+\lfloor C^*/\varepsilon \rfloor}\right)$ With equal costs, UCS behaves like BFS.

Depth-first search

Expand the **deepest** unexpanded node: the frontier is a LIFO stack. It is usually written **recursively**, as backtracking, storing only the current path:

```
function DFS(problem, node):
    if problem.is-goal(node.state): return node
    for each child in EXPAND(problem, node):
        if child.state is not on the path to node:
            result ← DFS(problem, child)
            if result ≠ failure: return result
    return failure
```

On the water jug problem, this version returns a **10-move** solution. Next week: **Prolog** runs every program exactly this way.

Depth-first search: guarantees

- **Not optimal**: on the water jug problem, 8 moves (graph search) or 10 moves (recursive version), against 6.
- **Completeness** depends on the variant:
  - graph search, with **reached**: complete in **finite** state spaces;
  - tree-like search: loops forever on a cycle, unless it checks the current path;
  - in **infinite** spaces, neither: it can descend an infinite branch forever (Knuth's 4).
- **Time** $O(b^m)$: terrible when $m \gg d$, and m may be ∞.
- **Space** $O(bm)$ for the tree-like variant, which stores only the current path and its siblings. **Linear memory** is the great advantage of DFS.

Depth-limited search

Depth-first search that treats nodes at depth ℓ as if they had no successors. Three possible results:

Result	Meaning
a goal node	a solution of depth at most ℓ
cutoff	no solution within depth ℓ, but the limit was reached: deeper ones may exist
failure	no solution at all: the whole cycle-free space was explored

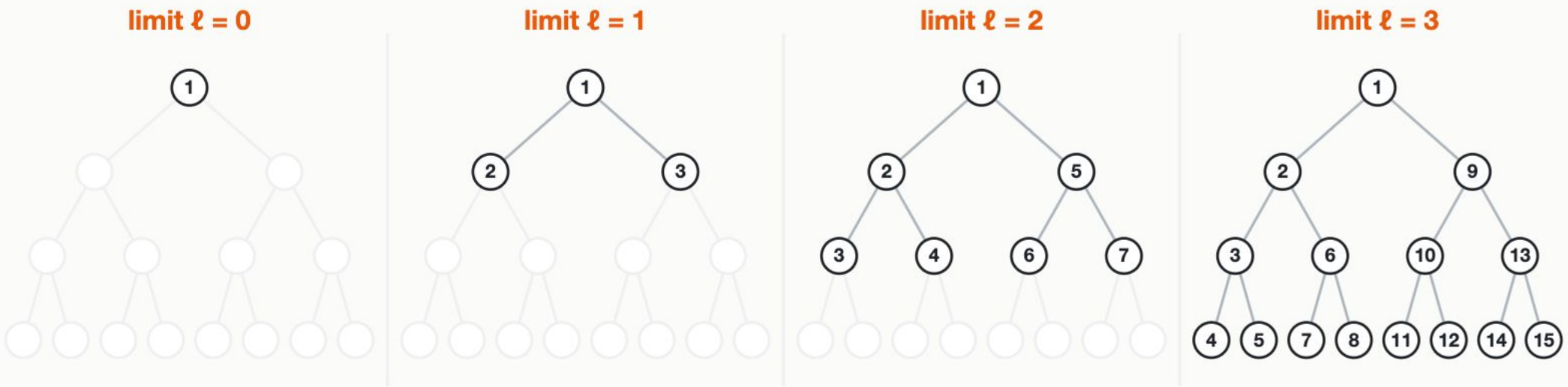
- Complete when $\ell \geq d$. Time $O(b^{\ell})$, space $O(b\ell)$.
- But d is usually unknown. In the water jug problem, every reachable state is within 7 moves of (0, 0), so $\ell = 7$ suffices.

Depth-limited search: the algorithm

```
function DEPTH-LIMITED-SEARCH(problem, ℓ):
    return RECURSE(problem, NODE(problem.initial-state), ℓ)

function RECURSE(problem, node, ℓ):
    if problem.is-goal(node.state): return node
    if node.depth = ℓ: return cutoff
    cutoff-occurred ← false
    for each child in EXPAND(problem, node):
        if child.state is on the path to node: continue
        result ← RECURSE(problem, child, ℓ)
        if result = cutoff: cutoff-occurred ← true
        else if result ≠ failure: return result
    if cutoff-occurred: return cutoff
    return failure
```

Iterative deepening search




```
function ITERATIVE-DEEPENING-SEARCH(problem):
    for ℓ = 0, 1, 2, …:
        result ← DEPTH-LIMITED-SEARCH(problem, ℓ)
        if result ≠ cutoff: return result
```

Isn't that wasteful?

A node of depth i is generated once in each iteration $\ell \geq i$, that is, d + 1 − i times:

$$N_{\mathrm{IDS}}(b,d) = \sum_{i=1}^{d} (d+1-i)\, b^i \qquad\qquad N_{\mathrm{BFS}}(b,d) = \sum_{i=1}^{d} b^i$$

For b = 10 and d = 5, iterative deepening generates only **11% more** nodes:

$$N_{\mathrm{IDS}}(10,5) = 123{,}450 \qquad\qquad N_{\mathrm{BFS}}(10,5) = 111{,}110$$

Proposition. $N_{\mathrm{IDS}}(b,d) \leq b^d \left(\frac{b}{b-1}\right)^2 = O(b^d)$

Most nodes are in the deepest level, so repeating the shallow levels is cheap.

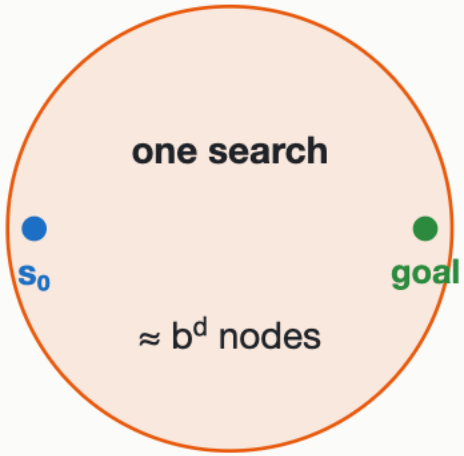
Iterative deepening: the best of both

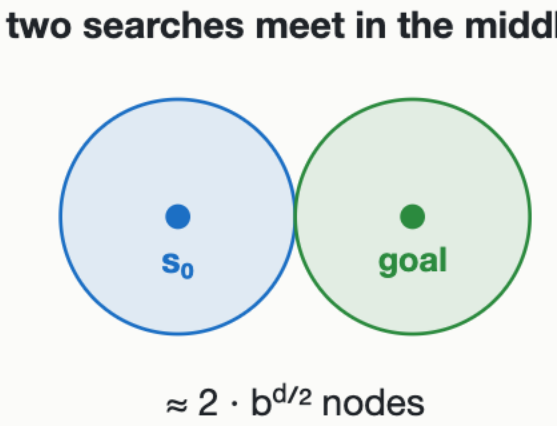
	BFS	DFS (tree-like)	Iterative deepening
Complete	yes	no	yes
Optimal for equal costs	yes	no	yes
Time	$O(b^d)$	$O(b^m)$	$O(b^d)$
Space	$O(b^d)$	O(bm)	O(bd)

(Completeness assumes a finite branching factor b.)

Rule of thumb: iterative deepening is the preferred blind method when the state space is large and the depth of the solution is unknown.

Bidirectional search





- Search forward from s_0 and backward from the goal; stop when the two frontiers meet.
- $2 \cdot b^{(d/2)} \ll b^d$: for b = 10 and d = 6, two thousand nodes instead of a million.
- It needs an explicit goal state, and a way to compute **predecessors**: the actions must be reversible, or their inverses computable.

Summary of blind search

Criterion	Breadth-first	Uniform-cost	Depth-first	Depth-limited	Iterative deepening	Bidirectional
Complete?	yes[1]	yes[1] [2]	no	no	yes[1]	yes[1] [4]
Optimal cost?	yes[3]	yes	no	no	yes[3]	yes[3] [4]
Time	$O(b^d)$	$O(b^{(1+\lfloor C^*/\varepsilon \rfloor)})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{(d/2)})$
Space	$O(b^d)$	$O(b^{(1+\lfloor C^*/\varepsilon \rfloor)})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{(d/2)})$

[1] if b is finite, and the state space is finite or has a solution · [2] if every action costs at least $\varepsilon > 0$ · [3] if all action costs are equal · [4] if both directions use breadth-first or uniform-cost search

Depth-first and depth-limited search refer to the tree-like versions, with linear memory.

Graph search or tree-like search?

	Graph search	Tree-like search	Tree-like + path check
Repeated states detected	all	none	cycles only
Extra memory	all reached states	none	none: the path is already stored
Main risk	memory	exponential redundant work, infinite loops	redundant paths

- BFS and UCS keep an exponential frontier anyway, so the **reached** set costs them little.
- DFS and iterative deepening use path checking, to keep their **linear memory**.

06

Search problems in Python

First attempt: recursive depth-first search

Everything lives in one function; the state is a pair (x, y).

```
def dfs_jugs(state=(0, 0), visited=None):
    if visited is None:
        visited = set()
    visited.add(state)
    x, y = state
    if x == 4 or y == 4:                          # goal: hard-wired
        return [state]
    t12, t21 = min(x, 3 - y), min(y, 5 - x)       # moves: hard-wired
    for nxt in [(5, y), (x, 3), (0, y), (x, 0),
                (x - t12, y + t12), (x + t21, y - t21)]:
        if nxt not in visited:
            rest = dfs_jugs(nxt, visited)
            if rest is not None:
                return [state] + rest
    return None
```

dfs_jugs() returns a solution with **10 moves**.

Second attempt: breadth-first search

The classic version with a queue (a **deque**); each entry is a whole path from the start.

```
def bfs_jugs():
    queue = deque([[(0, 0)]])                    # each entry: a path from the start
    visited = {(0, 0)}
    while queue:
        path = queue.popleft()
        x, y = path[-1]
        if x == 4 or y == 4:                     # goal: hard-wired
            return path
        t12, t21 = min(x, 3 - y), min(y, 5 - x)  # moves: hard-wired
        for nxt in [(5, y), (x, 3), (0, y), (x, 0),
                    (x - t12, y + t12), (x + t21, y - t21)]:
            if nxt not in visited:
                visited.add(nxt)
                queue.append(path + [nxt])
    return None
```

bfs_jugs() returns the shortest solution, with **6 moves**.

What is wrong with these programs?

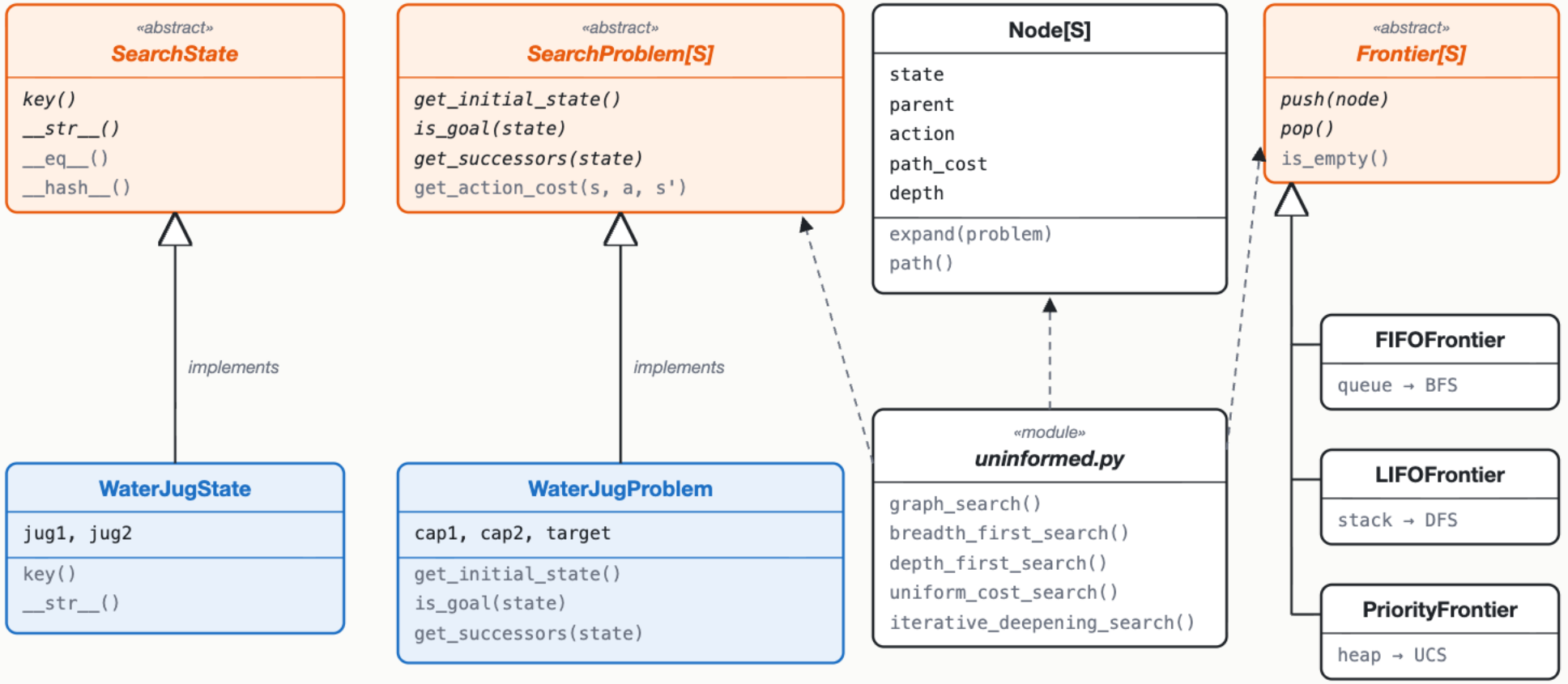
- The **problem** is written twice. The initial state, the moves and the goal are buried inside both functions: changing the jugs to 7 and 4 liters means editing both, in several places.
- The **search strategy** cannot be reused. Solving the 8-puzzle with breadth-first search means writing breadth-first search again, from scratch.
- Every **bug** must be fixed in every copy, and every copy can introduce new ones.

> **Our design goal:** separate *what the problem is* from *how we search*, so that each is written once.

Write each algorithm once

- We want **one breadth_first_search** that solves the jugs, Hanoi and the 8-puzzle, unchanged.
- **Program to an interface**: algorithms only know an abstract **SearchProblem**; each concrete problem implements it.
- Python's **abstract base classes** (**abc**) enforce the interface: a subclass that forgets a required method cannot be instantiated.
- **Generics** (**Generic[S]**) let a type checker verify that a problem and its states fit together.
- Four abstractions: **states**, **problems**, **nodes** and **frontiers**.

The design



italic = abstract: subclasses must implement it

States are values

```
class SearchState(ABC):
    """A configuration of a search problem. Never modify a state after creating it."""

    @abstractmethod
    def key(self) -> Hashable:
        """The data that identifies this configuration, e.g. a tuple."""

    @abstractmethod
    def __str__(self) -> str: ...

    def __eq__(self, other: object) -> bool:
        return type(self) is type(other) and self.key() == other.key()

    def __hash__(self) -> int:
        return hash(self.key())
```

Subclasses provide only **key()** and **__str__**. Equality and hashing are written **once**, and are always consistent: the **template method** pattern.

Why states must be values

```
class Plain:                              # no __eq__ or __hash__: equality is identity
    def __init__(self, a, b):
        self.a, self.b = a, b

Plain(2, 3) == Plain(2, 3)                # False
len({Plain(2, 3), Plain(2, 3)})           # 2: a "visited" set never recognizes a state

s = WaterJugState(0, 0)
visited = {s}
s.jug1 = 5                                # modifying a state that is stored in a set
s in visited                              # False
WaterJugState(0, 0) in visited            # False: the set is now corrupted
```

The rule: **get_successors builds new states**. It never modifies the state it receives.

The problem interface

```python
S = TypeVar("S", bound=SearchState)

class SearchProblem(ABC, Generic[S]):
    @abstractmethod
    def get_initial_state(self) -> S: ...

    @abstractmethod
    def is_goal(self, state: S) -> bool: ...

    @abstractmethod
    def get_successors(self, state: S) -> Iterable[tuple[S, str]]:
        """Pairs (next_state, action) for every action applicable in state."""

    def get_action_cost(self, state: S, action: str, next_state: S) -> float:
        return 1    # a hook: override it for non-unit costs
```

- **Abstract** methods: every problem *must* implement them. A **hook** has a default behavior; a problem *may* override it.

Why Generic[S]?

Without it, a concrete problem has to narrow the parameter type:

```
class SearchProblem(ABC):
    def is_goal(self, state: SearchState) → bool: ...

class WaterJugProblem(SearchProblem):
    def is_goal(self, state: WaterJugState) → bool: ...
```

```
error: Argument 1 of "is_goal" is incompatible with supertype "SearchProblem";
       supertype defines the argument type as "SearchState"
note:  This violates the Liskov substitution principle
```

- A subclass must accept **every** argument its parent accepts: parameter types are **contravariant**.
- With **SearchProblem[S]**, the state type is a **parameter**: **WaterJugProblem(SearchProblem[WaterJugState])** is consistent, and a type checker such as mypy verifies it.

Water jugs: the state

```
class WaterJugState(SearchState):
    """Contents of the two jugs, in liters."""

    def __init__(self, jug1: int, jug2: int) → None:
        self.jug1 = jug1
        self.jug2 = jug2

    def key(self) → tuple[int, int]:
        return (self.jug1, self.jug2)

    def __str__(self) → str:
        return f"({self.jug1}, {self.jug2})"
```

- A state holds **only what changes**. Capacities and target belong to the **problem**.
- Storing capacities in every state would duplicate them, and would make states of different puzzles compare equal.

Water jugs: the problem

```
class WaterJugProblem(SearchProblem[WaterJugState]):
    """Measure exactly `target` liters with two jugs (by default: 5, 3 and 4)."""

    def __init__(self, cap1: int = 5, cap2: int = 3, target: int = 4) -> None:
        self.cap1, self.cap2, self.target = cap1, cap2, target

    def get_initial_state(self) -> WaterJugState:
        return WaterJugState(0, 0)

    def is_goal(self, state: WaterJugState) -> bool:
        return self.target in (state.jug1, state.jug2)
```

One class, a whole family of puzzles: **WaterJugProblem(6, 4, 3)** is the impossible one.

Water jugs: the successors

```
def get_successors(self, state: WaterJugState) → list[tuple[WaterJugState, str]]:
    a, b = state.jug1, state.jug2
    t12 = min(a, self.cap2 - b)  # liters poured from jug1 into jug2
    t21 = min(b, self.cap1 - a)  # liters poured from jug2 into jug1
    moves = [
        (WaterJugState(self.cap1, b), "fill jug1"),
        (WaterJugState(a, self.cap2), "fill jug2"),
        (WaterJugState(0, b), "empty jug1"),
        (WaterJugState(a, 0), "empty jug2"),
        (WaterJugState(a - t12, b + t12), "pour jug1 into jug2"),
        (WaterJugState(a + t21, b - t21), "pour jug2 into jug1"),
    ]
    return [(s, action) for s, action in moves if s ≠ state]
```

- The amount poured is a formula, min(x, b − y): no loop needed. Moves that change nothing are filtered out.

The interface is enforced

```
class WaterJugProblem(SearchProblem[WaterJugState]):
    def get_initial_state(self) → WaterJugState: ...
    def get_successors(self, state: WaterJugState) → list[tuple[WaterJugState, str]]: ...
    # is_goal forgotten

WaterJugProblem()
```

```
TypeError: Can't instantiate abstract class WaterJugProblem without an
implementation for abstract method 'is_goal'
```

The error appears when the object is **created**, before any search runs (message as printed by Python 3.12).

Nodes in Python

A node of the search tree is a small dataclass; **expand** turns the successors of its state into child nodes:

```
@dataclass(eq=False)
class Node(Generic[S]):
    state: S
    parent: Optional[Node[S]] = None
    action: Optional[str] = None
    path_cost: float = 0
    depth: int = 0

    def expand(self, problem: SearchProblem[S]) → Iterator[Node[S]]:
        for next_state, action in problem.get_successors(self.state):
            cost = problem.get_action_cost(self.state, action, next_state)
            yield Node(next_state, self, action, self.path_cost + cost, self.depth + 1)
```

node.path() follows the parents back to the root; **node.actions()** gives the solution.

Graph search in Python

```
def graph_search(problem: SearchProblem[S],
                 frontier: Frontier[S]) → Optional[Node[S]]:
    """Generic graph search. The frontier decides the order of expansion."""
    root = Node(problem.get_initial_state())
    frontier.push(root)
    reached: dict[S, Node[S]] = {root.state: root}
    while not frontier.is_empty():
        node = frontier.pop()
        if problem.is_goal(node.state):
            return node
        for child in node.expand(problem):
            s = child.state
            if s not in reached or child.path_cost < reached[s].path_cost:
                reached[s] = child
                frontier.push(child)
    return None
```

The pseudocode, line by line. **reached** maps each state to the best node found for it so far.

Three strategies, one function

```
def breadth_first_search(problem):
    return graph_search(problem, FIFOFrontier())
def depth_first_search(problem):
    return graph_search(problem, LIFOFrontier())
def uniform_cost_search(problem):
    return graph_search(problem, PriorityFrontier(lambda n: n.path_cost))
```

In OO terms, the **strategy pattern**: the frontier object decides the order of expansion. Week 4 adds greedy search and A* by passing other priority functions.

Frontiers

```python
class Frontier(ABC, Generic[S]):
    @abstractmethod
    def push(self, node: Node[S]) -> None: ...

    @abstractmethod
    def pop(self) -> Node[S]: ...

    @abstractmethod
    def __len__(self) -> int: ...

    def is_empty(self) -> bool:
        return len(self) == 0
```

```python
class FIFOFrontier(Frontier[S]):
    def __init__(self) -> None:
        self._items: deque[Node[S]] = deque()

    def push(self, node: Node[S]) -> None:
        self._items.append(node)

    def pop(self) -> Node[S]:
        return self._items.popleft()

    def __len__(self) -> int:
        return len(self._items)
```

LIFOFrontier is the same with a list and **pop()**; **PriorityFrontier** keeps a binary heap ordered by a key function.

Running it on the water jugs

```
>>> node = breadth_first_search(WaterJugProblem())
>>> " → ".join(str(n.state) for n in node.path())
'(0, 0) → (5, 0) → (2, 3) → (2, 0) → (0, 2) → (5, 2) → (4, 3)'
>>> node.actions()[:2]
['fill jug1', 'pour jug1 into jug2']
>>> depth_first_search(WaterJugProblem()).depth
8
>>> print(breadth_first_search(WaterJugProblem(6, 4, 3)))
None
```

- Breadth-first search confirms that **6 moves** is the minimum; depth-first search returns 8 moves.
- On **WaterJugProblem(6, 4, 3)**, both return **None** after exploring every reachable state.

Depth-limited search in Python

```
def depth_limited_search(problem, limit):
    def recurse(node):
        if problem.is_goal(node.state):
            return node
        if node.depth == limit:
            return CUTOFF
        cutoff_occurred = False
        for child in node.expand(problem):
            if node.on_path(child.state):
                continue
            result = recurse(child)
            if result is CUTOFF:
                cutoff_occurred = True
            elif result is not None:
                return result
        return CUTOFF if cutoff_occurred else None

    return recurse(Node(problem.get_initial_state()))
```

Iterative deepening in Python

```
class Cutoff(Enum):
    CUTOFF = "cutoff"

CUTOFF: Final = Cutoff.CUTOFF

def iterative_deepening_search(problem):
    for limit in count():                          # 0, 1, 2, ...
        result = depth_limited_search(problem, limit)
        if result is not CUTOFF:
            return result
```

CUTOFF is the only member of an **Enum**, so a type checker can tell it apart from a node and from **None**, the Python version of failure.

Summary

- AI is broader than machine learning: we study **rational agents**, with symbolic and statistical tools.
- A **search problem** has states, an initial state, actions, a transition model, a goal test and costs. Its state space is an implicit graph, often huge or infinite.
- **Modeling** matters. The right states make a problem tractable (8 queens), and **invariants** explain what search can and cannot find (jugs, 8-puzzle).
- **Blind search**: BFS (optimal for equal costs, exponential memory), UCS (arbitrary positive costs), DFS (linear memory, few guarantees), and **iterative deepening** (the guarantees of BFS with the memory of DFS).
- **OO design**: abstract classes enforce the interface, generics keep problems and states consistent, and the frontier is a pluggable strategy.

In the lab

1. Complete **WaterJugProblem**. Run BFS, DFS and iterative deepening; compare the solutions and the number of nodes expanded.
2. Model **Knuth's 4**. Which strategies can find 5? What must you restrict, and why?
3. Model **n-queens**, placing one queen per column, and solve it for n = 8 with DFS and with BFS. Which strategy suits this problem better, and why?
4. Find the way out of a **maze**, with DFS and with BFS.

Next: logic programming

- A Prolog program is a set of facts and rules; running a query is a **search for a proof**.
- Prolog explores that search space with **depth-first search and backtracking**, today's algorithm.
- We will see what this implies: very short programs for search problems, and programs that never terminate.

Reading: Russell and Norvig, *Artificial Intelligence: A Modern Approach*, 4th edition, chapters 1–2 and sections 3.1–3.4.